waveeProjectBW - Dokumentacja Techniczna FULL

Wersja scalona wszystkich podstron do wydruku PDF.

01_architektura_i_przeplyw.html

01. Architektura i przeplyw danych

Tu jest rozpisane krok po kroku jak request idzie przez caly system.

1) Co to jest za typ aplikacji

waveeProjectBW to klasyczny uklad klient-serwer. Klientem jest frontend React, serwerem backend Express. Dodatkowo backend gada ze Spotify API i z baza PostgreSQL przez Prisma.

- Frontend odpowiada za UI i interakcje usera.
- Backend odpowiada za auth, logike biznesowa i zapis danych.
- Baza przechowuje userow, recenzje, playlisty i inne dane trwale.

2) Przeplyw requestu - prosty scenariusz

1. User otwiera strone i np. szuka piosenki.
2. Frontend wysyla GET na `/api/spotify/search?q=...`.
3. Backend sprawdza autoryzacje przez middleware.

4. Backend pobiera dane ze Spotify (lub mockData w demo).
5. Backend zwraca JSON.
6. Frontend renderuje wynik jako karty utworow.

3) Przeplyw auth (normalny i demo)

Wariant A: Spotify OAuth

1. Frontend odpytuje `/api/auth/login` po URL logowania.
2. Backend zwraca URL do Spotify accounts.
3. User loguje sie w Spotify i wraca na `/api/auth/callback`.
4. Backend wymienia code na token i robi JWT cookie.

Wariant B: Demo

1. Frontend wywoluje `/api/auth/demo-login`.
2. Backend tworzy/odswieza usera demo i ustawia cookie.
3. Aplikacja dziala na mockData bez prawdziwych tokenow Spotify.

4) Najwazniejsze punkty architektury

Obszar	Plik	Znaczenie
Start serwera	`backend/src/index.js`	Tu lacza sie wszystkie trasy i middleware.
Start frontu	`frontend/src/main.tsx`	Tu startuje app i provider cache.
Routing frontu	`frontend/src/App.tsx`	Tu sa trasy publiczne i chronione.

Obszar	Plik	Znaczenie
API klient	`frontend/src/lib/api.ts`	Jedno miejsce do requestow i odswiezania sesji.

Nastepna podstrona: 02. Backend rdzen i middleware

02_backend_rdzen.html

02. Backend rdzen i middleware

Tutaj jest analiza glownego startu serwera oraz auth middleware.

Plik: backend/src/index.js

To jest centralny punkt backendu. On robi 6 krytycznych rzeczy:

1. Laduje konfiguracje ze zmiennych srodowiskowych.
2. Uruchamia CORS i whitelist originow.
3. Uruchamia parser JSON i obsluge cookie.
4. Podpina route auth, user, reviews, spotify, playlists, recommendations.
5. Daje endpoint zdrowia `/api/health`.
6. Uruchamia globalny handler bledow.

```
app.use('/api/auth', authRoutes)
app.use('/api/user', userRoutes)
app.use('/api/reviews', reviewsRoutes)
app.use('/api/spotify', spotifyRoutes)
app.use('/api/playlists', playlistsRoutes)
app.use('/api/recommendations', recommendationsRoutes)
```

Plik: backend/src/middleware/auth.js

Ten middleware pilnuje, zeby request mial wazna sesje. Najpierw bierze token, potem go sprawdza.

```
const token = req.cookies.token || req.headers.authorization?.split(' ')[1]
if (!token) return res.status(401).json({ error: 'Authentication required' })

const decoded = jwt.verify(token, process.env.JWT_SECRET)
req.userId = decoded.userId
req.spotifyAccessToken = decoded.spotifyAccessToken
```

Co sie dzieje gdy JWT wygasnie

- Middleware probuje odczytac token mimo expiracji.
- Jesli ma refresh token Spotify, robi odswiezenie access tokena.
- Wystawia nowe cookie JWT i request idzie dalej bez wyrzucania usera.

Plik: backend/src/lib/prisma.js

Singleton Prisma. Dzieki temu przy hot-reload nie tworzy sie duzo klientow DB i nie ma glupich problemow z polaczeniami.

```
const globalForPrisma = globalThis
export const prisma = globalForPrisma.prisma ?? new PrismaClient()
if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma
```

Plik: backend/src/lib/spotify.js (rdzen integracji)

To jest najwiekszy plik logiczny backendu. Odpowiada za:

- zbudowanie URL logowania Spotify,
- wymiane code na tokeny,

- refresh tokena,
- search i track details,
- audio features z fallbackiem na `null`,
- rekomendacje z nowa strategia (bez starych deprecated endpointow).

Najwazniejsze: projekt juz jest przygotowany na zmiany Spotify po 2024, wiec nie opiera sie na wycofanym endpoint `/recommendations` w starej formie.

Nastepna podstrona: 03. Backend route i API

03_backend_route_api.html

03. Backend route i API

Tu rozkladam endpointy po kolei i opisuje co robia w praktyce.

1) Auth routes - backend/src/routes/auth.js

Endpoint	Metoda	Opis
/api/auth/register	POST	Rejestracja email + haslo (lokalnie)
/api/auth/local-login	POST	Login lokalny
/api/auth/login	GET	Pobranie URL do Spotify OAuth
/api/auth/demo-login	POST	Szybki login demo
/api/auth/callback	GET	Callback Spotify
/api/auth/me	GET	Pobranie zalogowanego usera
/api/auth/refresh	POST	Odswiezenie tokenu
/api/auth/logout	POST	Wylogowanie

```
const localAuthSchema = z.object({
  email: z.string().email().max(120),
```

```
  password: z.string().min(6).max(72),
})
```

2) Spotify routes - backend/src/routes/spotify.js

- `GET /api/spotify/search` - szukanie piosenek.
- `GET /api/spotify/track/:trackId` - szczegoly utworu + audio features.
- `GET /api/spotify/audio-features/:trackId` - tylko audio features.
- `GET /api/spotify/top/tracks` i `/top/artists` - top usera.
- `GET /api/spotify/genres` - lista gatunkow.

Dla usera demo backend nie pyta Spotify i zwraca dane z `mockData.js`.

3) Reviews routes - backend/src/routes/reviews.js

Najmocniejsza rzecz: recenzja idzie jako upsert, czyli jak ten user juz recenzowal ten track, to idzie update, a nie drugi rekord.

```
await prisma.review.upsert({
  where: { userId_trackId: { userId: req.userId, trackId } },
  update: { rating, content },
  create: { userId: req.userId, trackId, rating, content }
})
```

- `GET /api/reviews/my` - recenzje usera z paginacja.
- `GET /api/reviews/track/:trackId` - feed recenzji utworu.
- `DELETE /api/reviews/:reviewId` - tylko wlasciciel moze usunac.

4) Playlists i recommendations

Playlists - backend/src/routes/playlists.js

- CRUD playlist
- dodawanie/usuwanie trackow
- sync playlisty do Spotify

Recommendations - backend/src/routes/recommendations.js

- `/similar/:trackId` - podobne utwory
- `/discover/:genre` - odkrywanie po gatunku

Nastepna podstrona: 04. Baza danych i Prisma

04_baza_danych_prisma.html

04. Baza danych i Prisma

W tej sekcji masz dokladny opis modeli, relacji i ograniczen.

Plik: backend/prisma/schema.prisma

Projekt pracuje na PostgreSQL. Prisma trzyma modele i pilnuje spojnosc relacji.

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

Model User

- `spotifyId` unikalne ID usera.
- `email` moze byc null (np. niektore profile).
- `favoriteGenres` tablica string.
- Relacje do `Review`, `Playlist`, `LikedTrack`.

Model Review

To nie jest tylko ocena. Tu jest tez snapshot metadanych utworu i cechy audio.

- `trackId`, `trackName`, `artistName`, `albumName`
- `rating` 0.5 do 5.0
- `tempo`, `key`, `energy`, `danceability`, `valence`
- `@@unique([userId, trackId])` blokuje duplikat tej samej recenzji usera na ten sam utwor

Model Playlist i PlaylistTrack

- Playlist ma nazwe, opis, wlasciciela i opcjonalne spotifyPlaylistId.
- PlaylistTrack trzyma pozycje w playliscie.
- `@@unique([playlistId, trackId])` ogranicza duplikaty trackow.

Model LikedTrack

To model pod funkcje rozszerzone (ulubione, rekomendacje personalne, profil gustu). Trzyma tez cechy audio i tablice gatunkow.

Wnioski z projektu bazy

1. Dane sa dobrze znormalizowane jak na ten poziom aplikacji.
2. Wazne ograniczenia unique chronia przed logicznymi duplikatami.
3. Relacje i cascade delete ulatwiaja porzadek przy usuwaniu usera.

Nastepna podstrona: 05. Frontend rdzen i komunikacja

05_frontend_rdzen.html

05. Frontend rdzen i komunikacja

To jest serce frontendu: start appki, routing, sesja i API client.

1) frontend/src/main.tsx

Tu rusza React i konfiguracja cache query. QueryClient trzyma dane z API i ogranicza liczbe zapytan do backendu.

```
export const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 1000 * 60 * 5,
      retry: 1,
    },
  },
})
```

2) frontend/src/App.tsx

Tu jest mapa wszystkich tras. Czesci prywatne sa opakowane przez ProtectedRoute.

- Publiczne: Home, Login

- Chronione: Dashboard, Search, Track, Reviews, Generator, Community, Profile

3) frontend/src/contexts/AuthContext.tsx

Context daje globalny stan zalogowania i usera. Przy starcie probuje pobrac `/auth/me`.

```
const { data } = await getCurrentUser()
setUser(data)
```

Przy logout czyści cache query przez `queryClient.clear()`, wiec nie ma starych danych po sesji.

4) frontend/src/lib/api.ts

To jeden wspolny klient HTTP. Najwazniejszy jest interceptor 401, ktory sam probuje odswiezyc sesje.

```
if (!originalRequest || status !== 401 || originalRequest._retry || shouldSkipR
  return Promise.reject(error)
}
originalRequest._retry = true
await api.post('/auth/refresh')
return api(originalRequest)
```

Dlaczego to jest dobre

- Nie trzeba recznie obslugiwac refresh w kazdej stronie.
- User rzadziej widzi logowanie od nowa.
- Logika API jest skupiona w jednym miejscu.

5) Layout i ProtectedRoute

- `Layout.tsx` daje wspolna nawigacje i stopke.
- `ProtectedRoute.tsx` blokuje wejscie bez sesji.

Nastepna podstrona: 06. Frontend strony i UI

06_frontend_strony_ui.html

06. Frontend strony i UI

Dokladny opis ekranow i komponentow, co user klika i co sie dzieje pod spodem.

Home i Login

- `pages/Home.tsx` ma przycisk Spotify i przycisk Demo.
- `pages/Login.tsx` odpytuje backend o URL logowania Spotify.

W obu ekranach jest fallback demo, wiec prezentacja projektu nie pada przez brak API key.

Dashboard

`pages/Dashboard.tsx` odpytuje:

- top tracks,
- top artists,
- statystyki recenzji,
- ostatnie recenzje usera.

Ekran ma przelacznik zakresu czasu, np. 4 tygodnie, 6 miesiecy, all-time.

Search i Track

Search

- formularz szukania po frazie,
- wyniki jako karty trackow,
- obsluga loading i error.

Track

- szczegoly utworu (artist, album, czas),
- audio features,
- formularz recenzji (gwiazdki + komentarz),
- lista recenzji i podobne utwory.

Reviews, Generator, Profile, Community

Reviews

Lista recenzji usera + usuwanie.

PlaylistGenerator

Dwa tryby: po gatunku albo po wybranym tracku. Ma usuwanie niepasujacych utworow i podmiane z puli zapasowej.

Profile

Edycja displayName, bio i favoriteGenres + statystyki.

Community

Szukanie userow i podglad profilu publicznego. W demo jest blokada tej funkcji.

Komponenty techniczne UI

- `AudioFeaturesDisplay.tsx` - paski energii, danceability itd.
- `StarRating.tsx` - gwiazdki z polowkami.
- `TrackCard.tsx` - karta utworu.
- `index.css` - globalny styl i scrollbar.

Nastepna podstrona: 07. Uruchomienie, bezpieczenstwo, testy

07_uruchomienie_bezpieczenstwo_testy.html

07. Uruchomienie, bezpieczenstwo i testy

Finalna sekcja: jak odpalic projekt, jak go testowac i na co uwazac.

1) Uruchomienie lokalne

Backend

```
cd backend
npm install
npm run db:generate
npm run db:push
npm run dev
```

Frontend

```
cd frontend
npm install
npm run dev
```

Zmienne .env (backend)

```
PORT=3001
CLIENT_URL=http://localhost:5173
```

```
DATABASE_URL=postgresql://USER:PASS@localhost:5432/waveeProjectBW
JWT_SECRET=twoj_super_sekret
SPOTIFY_CLIENT_ID=...
SPOTIFY_CLIENT_SECRET=...
SPOTIFY_REDIRECT_URI=http://localhost:3001/api/auth/callback
```

2) Checklista testow recznych

1. Login demo i wejscie do dashboard.
2. Wyszukanie tracka i wejscie do szczegolow.
3. Dodanie recenzji 4.5 i sprawdzenie listy recenzji.
4. Generator playlisty po gatunku i po tracku.
5. Usuniecie recenzji i weryfikacja odswiezenia danych.
6. Wylogowanie i proba wejscia na strone chroniona.

3) Bezpieczenstwo - co jest i co warto dopchnac

Juz jest dobrze

- JWT i middleware auth.
- Walidacja danych przez Zod.
- Kontrola wlasciciela zasobu (playlista/recenzja).
- Obsluga bledow Spotify z czytelnym komunikatem.

Warto dorobic w kolejnym kroku

- Rate limiting na endpointy logowania i search.
- `secure: true` dla cookie na produkcji.
- Testy automatyczne backend route.
- Monitoring i centralny logger.

4) Co powiedziec przy obronie projektu

1. Pokazac tryb demo jako zabezpieczenie prezentacji.
2. Pokazac upsert recenzji i unikalnosc danych.
3. Pokazac interceptor 401 i auto-refresh sesji.
4. Pokazac jak frontend i backend dzialaja oddzielnie ale spojnie.

Taki flow zwykle robi bardzo dobre wrazenie, bo pokazuje nie tylko UI ale tez ogarnieta logike.

Podpis autorow dokumentacji: Aleksander Baran, Franciszek Wawrzeń

Koniec wersji wielostronicowej HTML.

Podpis autorow: Aleksander Baran, Franciszek Wawrze

Data: 17.03.2026